



School of Computing

Master of Computer Applications (MCA)

Course Name : Design and Analysis of Algorithms

Course Code : TMC-301

Semester : III

Pre-requisite : Advance Data Structures

L	T	P	Credits
3	0	0	3

Course Learning Objectives:

- To understand the fundamental concepts of algorithm design, asymptotic analysis, recursion, recurrence relations, and computational complexity.
- To apply Divide and Conquer techniques and efficient sorting algorithms to design and solve computational problems.
- To apply Greedy and Dynamic Programming paradigms to develop efficient solutions for optimization and graph-related problems.
- To apply Backtracking, Branch and Bound, and String Matching techniques to solve combinatorial optimization and pattern-matching problems.
- To design, implement, and evaluate an algorithm-based project by selecting appropriate algorithm design paradigms to solve real-world computational problems and effectively communicate the results through technical documentation and presentation.

Course Outcomes:

After completion of the course, the students will be able to:

CO	Statement	Bloom's Level	Skill
CO1	Analyze the fundamental concepts of algorithm design, asymptotic notations, recurrence relations, and evaluate the time and space complexity of algorithms for solving computational problems.	L4 – Analyze	Analytical Thinking, Problem Analysis
CO2	Apply Divide and Conquer techniques to design and implement efficient algorithms for computational problems and analyze their performance.	L3 – Apply	Problem Solving, Algorithm Design, Programming
CO3	Design and implement efficient solutions using Greedy and Dynamic Programming paradigms for optimization and graph-related problems, and evaluate their effectiveness.	L5 – Evaluate	Algorithm Design, Optimization, Critical Thinking
CO4	Apply Backtracking, Branch and Bound, and String Matching techniques to develop efficient solutions for complex computational problems and analyze their efficiency.	L3 – Apply	Programming, Problem Solving, Algorithm Implementation
CO5	Design, implement, evaluate, and present a project based on algorithm design techniques by selecting appropriate paradigms to solve real-world computational problems while considering correctness, efficiency, scalability, and practical applicability.	L6 – Create	Project Development, Innovation, Teamwork, Communication, Decision Making



School of Computing

Master of Computer Applications (MCA)

Details of the Course:

Unit 1: Algorithm Foundations and Complexity Analysis

Contact Hours/Teaching Hours: 9

Introduction: Fundamentals of Algorithms, Characteristics of Algorithms, Algorithm Specification, Pseudo code Conventions.

Performance Analysis: Space complexity, Time complexity, Analysis of Iterative Algorithms, Best, Average, and Worst-Case Analysis, Asymptotic notation (O , Ω , Θ), Growth of Functions, Practical complexities, Performance measurement

Recursive Algorithm: Introduction to Recursion, Recursive Algorithm Design, Greatest Common Divisor (Euclid's Algorithm), Towers of Hanoi, Permutation Generator.

Recurrence Relations: Introduction to Recurrence Relations, Formulation of Recurrence Relations for Recursive Algorithms, Solving Recurrence Relations using: Iteration (Substitution) Method, Recursion Tree Method and Master Theorem.

Self-Learning Hours = 6

Students shall practice writing algorithms and pseudo code, analyze the time and space complexity of algorithms using asymptotic notations, solve recurrence relations using different methods, and implement recursive algorithms such as Euclid's Algorithm and Towers of Hanoi in C.

Team Activity Hours = 3

Students shall work in groups to compare recursive and iterative approaches, analyze the efficiency of different algorithms using asymptotic notations, and present solutions to recurrence relations and real-world algorithmic problems using appropriate design techniques.

Unit 2: Divide and Conquer and Efficient Sorting Algorithms

Contact Hours/Teaching Hours: 9

Divide and conquer: General method, Performance Analysis of Binary search, Merge sort and Quicksort, Randomized Quick Sort and Performance Analysis, Finding the maximum and minimum, Selection algorithms (Kth smallest)

Non Comparison Sorting: Counting and Radix Sort

Self-Learning Hours = 6

Students shall implement Divide and Conquer algorithms including Binary Search, Merge Sort, Quick Sort, Randomized Quick Sort, Maximum–Minimum, and Selection algorithms, analyze their time complexity, and compare them with non-comparison sorting techniques such as Counting Sort and Radix Sort.

Team Activity Hours = 3

Students shall work in groups to compare Divide and Conquer algorithms based on efficiency and scalability, analyze the performance of different sorting and selection algorithms, and present suitable algorithmic approaches for solving real-world computational problems.

Unit 3: Greedy Paradigm and Graph Algorithms

Contact Hours/Teaching Hours: 9

The Greedy Method: General Method, Fractional Knapsack Problem, Optimal Merge Pattern, Job Sequencing with Deadlines, Activity Selection, Minimum-Cost Spanning Trees, Prim's Algorithm, Kruskal's Algorithm, Topological Sorting, Single-Source Shortest Paths: Dijkstra's Algorithm.

Self-Learning Hours = 6



School of Computing

Master of Computer Applications (MCA)

Students shall implement Greedy algorithms for optimization problems including Fractional Knapsack, Job Sequencing with Deadlines, Activity Selection, Minimum Spanning Trees (Prim's and Kruskal's algorithms), Topological Sorting, and Dijkstra's algorithm, and analyze their correctness and efficiency.

Team Activity Hours = 3

Students shall work in groups to compare Greedy algorithm strategies for different optimization and graph problems, justify algorithm selection based on problem characteristics, and present real-world applications of minimum spanning trees, shortest path algorithms, and scheduling techniques.

Unit 4: Dynamic Programming and String Matching Algorithms

Contact Hours/Teaching Hours: 9

Dynamic Programming: The general method, Multistage graphs, All pairs shortest paths, Single-source shortest paths, Optimal Binary Search Trees, 0/1 Knapsack, Longest Common Subsequence (LCS), Matrix Chain Multiplication

Introduction to String Matching Algorithms: Naïve String Matching, Knuth–Morris–Pratt (KMP), and Rabin–Karp Algorithm.

Self-Learning Hours = 6

Students shall implement Dynamic Programming algorithms including Multistage Graphs, Shortest Path Problems, Optimal Binary Search Trees, 0/1 Knapsack, Longest Common Subsequence (LCS), and Matrix Chain Multiplication, and practice the implementation and performance analysis of string matching algorithms such as Naïve String Matching, Knuth–Morris–Pratt (KMP), and Rabin–Karp.

Team Activity Hours = 3

Students shall work in groups to compare Dynamic Programming solutions with other algorithm design paradigms, analyze the efficiency of shortest path and optimization algorithms, and present the working and applications of string matching algorithms in real-world scenarios such as text processing, and information retrieval.

Unit 5: Backtracking and Branch-and-Bound Techniques

Contact Hours/Teaching Hours: 9

Backtracking: The general method, Sum of Subsets, The 8-queens problem, Graph coloring, Hamiltonian cycles, Knapsack problem.

Branch-and-bound: The method, Least cost (LC) search, Control abstractions for LC-search, Bounding, FIFO branch-and-bound, LC branch-and-bound, 0/1knapsack problem, Travelling sales person problem.

Self-Learning Hours = 6

Students shall implement Backtracking algorithms for combinatorial problems including Sum of Subsets, 8-Queens, Graph Coloring, Hamiltonian Cycles, and Knapsack, and practice Branch-and-Bound techniques for solving optimization problems such as the 0/1 Knapsack Problem and Travelling Salesperson Problem using FIFO and Least-Cost (LC) search strategies.

Team Activity Hours = 3

Students shall work in groups to compare Backtracking and Branch-and-Bound approaches based on search strategy, pruning techniques, computational efficiency, and solution quality, and present suitable algorithmic solutions for combinatorial optimization problems and real-world applications.



School of Computing

Master of Computer Applications (MCA)

Course Articulation Matrix

CO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PSO1	PSO2	PSO3
CO1	3	3		2				2	2		
CO2	2	3	3	3				2	3		2
CO3	2	3	3	2	2			2	3		2
CO4	2	3	3	3				2	3		2
CO5	2	3	3	3	3	3	2	3	3	2	3
Average	2.2	3.0	3.0	2.6	1.0	0.6	0.4	2.2	2.8	0.4	1.8

Text Books:

1. **Anany Levitin**, *Introduction to the Design and Analysis of Algorithms*, **3rd Edition**, Pearson Education, India, **2017** (Indian Reprint).
2. **Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein**, *Introduction to Algorithms*, **4th Edition**, MIT Press, Cambridge, Massachusetts, USA, **2022**.

Reference Books:

1. **Steven S. Skiena**, *The Algorithm Design Manual*, **3rd Edition**, Springer Nature, Cham, Switzerland, **2020**.
2. **Ellis Horowitz, Sartaj Sahni, and Sanguthevar Rajasekaran**, *Fundamentals of Computer Algorithms*, **2nd Edition**, Universities Press (India) Pvt. Ltd., Hyderabad, India, **2008** (Reprint).
3. **Amrinder Arora**, *Analysis and Design of Algorithms*, **2nd Edition**, McGraw Hill Education (India) Pvt. Ltd., New Delhi, India, **2015**.
4. **Harsh Bhasin**, *Algorithms: Design and Analysis*, **2nd Edition**, Oxford University Press, New Delhi, India, **2018**.

Online Resources:

1. **NPTEL – Design and Analysis of Algorithms**
<https://nptel.ac.in/>
2. **MIT OpenCourseWare – Introduction to Algorithms**
<https://ocw.mit.edu/courses/6-006-introduction-to-algorithms/>
3. **VisuAlgo – Algorithm Visualizations**
<https://visualgo.net/>
4. **GeeksforGeeks – Algorithms**
<https://www.geeksforgeeks.org/fundamentals-of-algorithms/>
5. **LeetCode – Algorithms Practice**
<https://leetcode.com/problemset/algorithms/>